Phase 2: Improving the Tree CRDT with Vector Clocks and Checkpointing

CS 403/534 - Distributed Systems
Course Project, Spring 2025-2026

May 1, 2026

Deadline: May 11, 23:55 (Turkish Time)

Abstract

Phase 2 extends the Phase 1 Tree CRDT along three axes. First, you will introduce a `Delete` operation alongside `Move` and replace the totally ordered Lamport clock with a partially ordered vector clock — the resulting concurrency generalises the underlying data structure from a tree into a directed acyclic graph (DAG) of versions, with concurrent `Move`/`Delete` pairs resolved via a *Move-Wins* policy. Second, you will discuss backward compatibility with Phase 1 theoretically in your report and implement forward-compatibility precautions for an as-yet-unknown Phase 3. Third, you will implement *checkpointing*: causal-stability-driven log compaction that lets each replica discard log entries it can prove no future delivery will invalidate. This document is written as a *delta* on the Phase 1 specification and should be read alongside it.

1 Introduction

Phase 1 of the project gave you a working Tree CRDT in which each replica holds a Lamport clock, a tree, and an operation log, and synchronises state with peers through a Publish-Subscribe channel using `MOVE` messages. Because Lamport timestamps form a total order on (t, i)-pairs (where t is the timestamp and i is the replica ID), Phase 1 needs no explicit conflict resolution: the *undo-do-redo* sequence is sufficient to reconcile any out-of-order delivery into the same final tree on every replica.

Phase 2 relaxes this assumption in two complementary ways. First, the operation set grows: alongside `Move`, now there exists `Delete`, a tombstone-style operation that marks a node as deleted from the tree. Second, the clock loses its total order: replacing the Lamport clock with a vector clock means that two operations may now be *concurrent*: neither happens-before the other.

Together, these changes mean that a node identifier c can simultaneously have several legitimate "versions" (e.g. two replicas concurrently moving the same node under different parents), or a `Move` happening concurrently with a `Delete` of the same node. Phase 1's single-value tree, in which each c had at

most one parent, can no longer represent this state without losing information; conceptually, the data structure underlying the CRDT generalises from a tree into a *directed acyclic graph (DAG)* of versions, and the user-facing notion of "the active tree" is recovered as a filtered subgraph of that DAG. This generalisation is one of the central conceptual changes of Phase 2. The one case in which an explicit conflict-resolution rule is required is a concurrent `Move`/`Delete` pair on the same child, which you are going to resolve in favour of `Move` via a *Move-Wins* policy; all other concurrent updates simply coexist as parallel versions.

Two implementation notes carry through. The choice of *how to store* the per-child set of versions internally — a dictionary of sets, a list, or any other container of `Node` objects — remains yours, exactly as in Phase 1: the spec below constrains the contract of the `Tree` class, but not its internal data structure. And the changes ripple through three layers: the `Clock` hierarchy gains a new concrete subclass, `VectorClock`, along with four comparison helpers in that subsclass; the `Tree` class generalises from single-value to per-child sets of versions, with filtering for tombstones and orphans; and the `Replica` class gains machinery to detect conflicts in the operation log, to selectively undo or rewind through a *tree snapshot*, and to compact its operation log once it can do so safely. The communication protocol changes only in spirit: you must broadcast *some* information about your most recent timestamps to your peers, but the precise wire format is one of the design choices you have to defend in your report under "backward compatibility."

2 Description

2.1 Vector Clocks

Each replica's logical clock is replaced by a vector clock. Concretely, the clock state held by replica i in a system of N replicas is a mapping

$$V_i \;:\; \{0, 1, \ldots, N-1\} \;\to\; \mathbb{Z}_{\geq 0},$$

typically realised in Python as a `dict[int, int]`. The update rule on a local event is $V_i[i] \mathrel{+}= 1$, and on receipt of a remote operation tagged with vector W the rule is

$$V_i[k] \;\leftarrow\; \max\bigl(V_i[k],\, W[k]\bigr) \quad \text{for all } k,$$

followed by the same self-tick $V_i[i] \mathrel{+}= 1$.

You should implement the vector clock as a new concrete subclass `VectorClock` of the `Clock` abstract base class from Phase 1 (see Section 2.5.1 for the required signature). The reason for keeping the `Clock` abstraction is that the rest of the library —in particular the `Replica` class— should ideally be agnostic to the concrete clock type.

A note on partial order. For two vector timestamps V and W with the same key set, define

$$\begin{aligned}
V \leq W \;&\iff\; \forall k:\; V[k] \leq W[k],\\
V < W \;&\iff\; V \leq W \;\wedge\; V \neq W,\\
V = W \;&\iff\; \forall k:\; V[k] = W[k],\\
V \parallel W \;&\iff\; \neg(V \leq W) \;\wedge\; \neg(W \leq V).
\end{aligned}$$

The last relation, *concurrency*, is the one that did not exist under Lamport clocks and which all of Phase 2's new logic is concerned with.

2.2 The `Delete` Operation

In Phase 1 the only operation supported by the Tree CRDT was `Move`. Phase 2 introduces a second operation, `Delete`, which marks a node as removed from the tree.

Rather than introducing a new payload class on the wire, `Delete` is encoded as a special case of `Move`: a `Move` operation whose metadata carries the reserved key `"status": "deleted"` acts as a deletion of the target child, while all other operations carry `"status": "active"` and continue to behave exactly as in Phase 1. This encoding keeps the wire format and the `MovePayload` class unchanged from Phase 1, and it makes the deletion's parent and metadata fields available to the conflict-resolution machinery (in particular to Move-Wins; see Section 2.3).

Because the system is a CRDT, a `Delete` cannot eagerly remove the targeted node version from the tree state at the moment it is applied: a peer may still concurrently issue a `Move` of the same child, and silently dropping the deleted version would lose the information needed to resolve that conflict deterministically across replicas. `Delete` therefore behaves as a *tombstone*, the deleted version is retained in the per-child set of versions, with `"status": "deleted"` signalling that it should not contribute to the user-facing "active tree." The mechanics of how the active subtree is computed from a per-child set of versions in the presence of tombstones are described in Section 2.3 (per-child resolution under concurrency) and Section 2.4 (the cascade by which descendants of a fully-tombstoned ancestor become orphaned).

Two responsibilities are split between the user and the library around the metadata of an operation:

- The *user* (i.e. the code in `main.py` that generates operations) is responsible for setting `metadata["status"]` to either `"active"` or `"deleted"` when calling `Replica.apply_local_move(...)`. The library does not infer this from any other field.
- The *library* is responsible for setting and updating `metadata["applied"]` — a Boolean recording whether the operation actually took effect on the tree state (as opposed to merely being inserted into the operation log because it lost a Move-Wins resolution; see Section 2.5.4). The user must not write to this key directly.

2.3 Multi-Value Tree State and the Move-Wins Concurrency Semantics

In Phase 1, the state of the tree at each replica was, conceptually, a set of (p, m, c) triples in which c was unique. In Phase 2 we lift this to a *multi-value* associative structure: the state is a mapping from each child ID c to a *set* of versions of that node, where each version remembers the replica that produced it and the timestamp at which it was produced. Concretely, each version is now a quintuple

$$(i,\ t,\ p,\ m,\ c),$$

i.e., the original Phase 1 triple augmented with the producer's replica ID i and the producer's timestamp t (an integer for Lamport, a `dict` for vector clocks).

We say that two `Move` operations *conflict* when their timestamps are concurrent ($V \parallel W$) *and* their child IDs are equal. Operations that are concurrent but target distinct children do *not* conflict and are simply applied independently.

When two `Move` operations conflict, the multi-value set for the affected child stores both versions side by side; this preserves information about the system's history that could not be reconciled otherwise. The *Move-Wins* policy concerns the special case in which one of the two conflicting operations is a deletion — represented by the metadata key `"status"` taking the value `"deleted"` (see Section 2.5.2) — and the other is not. The policy is:

- If an incoming operation is a `Delete` that is concurrent with an existing `Move` on the same child, the `Delete` *loses*: it is recorded in the operation log but is not applied to the tree.
- If an incoming operation is a `Move` that is concurrent with an existing `Delete` on the same child, the `Delete` *loses retroactively*: its applied flag is flipped to `False`, and the tree is rebuilt to reflect the new state.
- If both conflicting operations are moves (or both are deletes), both versions are kept in the multi-value set without any further preference.

The consequence implied by the Move-Wins concurrency semantics is that an operation's effect on the tree is no longer a pure function of its own arguments: it depends on what is already in the multi-value set, and on what is already in the operation log. As a result, *both* the `Tree` class and the `Replica` class should be participating in the resolution, as they do so against different sources of information:

- The `Tree` class should resolve the incoming candidate against the multi-value versions *currently held for the same child c in the tree*. This resolution is local to a single call to `move()` and runs every time an operation is freshly applied to the tree — including every redo step inside an undo-do-redo sequence.
- The `Replica` class should resolve the incoming operation against the *operation log*, which may contain entries that have *already been applied* to the tree and whose effects Move-Wins now requires to be retracted. The retraction is performed by flipping the past entry's `"applied"` flag and rebuilding the tree from a snapshot.

The tree-level resolution is necessary because, in steady state, every new version must dominate, be dominated by, or coexist with the existing multi-value set, and that comparison cannot be done from the log alone. The replica-level resolution is necessary because the log contains operations whose effect on the tree may pre-date the incoming operation; without inspecting the log, the replica cannot detect that a past `Delete` should now lose to an incoming concurrent `Move`. For a concretization of the two layers, see the Sections 2.5.3 and 2.5.4.

2.4 Orphaned Nodes

Multi-value semantics introduces a subtle case that did not exist under single-value semantics. Consider a node whose multi-value set contains *only* versions whose status is `"deleted"`. Any descendants of that node, regardless of their own status, are now logically detached from the tree — there is no path from them up to the root through alive ancestors. We call such descendants *orphaned*.

You are required, when answering "is this node currently in the tree?", to filter orphans out. A reasonable detection algorithm is to run a breadth- or depth-first search from the candidate version upward through the multi-value parents, and to consider the candidate alive iff at least one such path reaches a live root version. The exact algorithm and its implementational details are up to you, but the semantics it must implement are fixed: a node is *active* iff (i) at least one of its versions has status `"active"`, *and* (ii) at least one such active version has a path to the root through nodes whose multi-value set is not entirely in the deleted state.

This requirement is what motivates the new `get_active(key)` method on `Tree` described in Section 2.5.3.

2.5 API Updates

This section enumerates the user-visible changes to the `tree-crdt` library. The signatures shown below are the *required* signatures; you may add helper methods and private fields as you see fit. Anything not mentioned is unchanged from Phase 1.

2.5.1 The `VectorClock` Class

Residing at *src/tree_crdt/clock/vector.py*, the `VectorClock` class is a concrete implementation of `Clock` that stores its timestamp as a dictionary $\{i \mapsto V_i\}$.

The fields of the class should be:

- `__id`: the integer ID of the owning replica.
- `__timestamp`: a `dict[int, int]` initialised to $\{0 : 0, 1 : 0, \ldots, N - 1 : 0\}$ where N is the number of replicas in the system.

The methods of the class are the abstract methods inherited from `Clock` (`__init__`, `id`, `timestamp`, `update`, `__str__`). Two contracts deserve emphasis:

- The constructor takes a second argument `max_id: int`, denoting the size of the replica set. This is the only signature change relative to Phase 1's `LamportClock`.
- The `timestamp` property must return a *deep copy* of the internal dictionary, not a reference to it; vector timestamps are mutable and accidental aliasing is a frequent source of bugs.

The `update(received)` method should accept either `None` (local event) or a `dict[int, int]` (remote event), perform the component-wise max merge in the remote case, and finish with the self-tick.

Static comparison helpers. In addition to the methods described above, you must equip the `VectorClock` class with four *static* methods that the rest of the library — the `Tree` and `Replica` classes, in particular — uses to query the partial order on vector timestamps defined in Section 2.1:

- `timestamp_le(lhs, rhs)` : returns `True` iff $lhs \leq rhs$.
- `timestamp_lt(lhs, rhs)` : returns `True` iff $lhs < rhs$.
- `timestamp_eq(lhs, rhs)` : returns `True` iff $lhs = rhs$.
- `timestamp_concurrent(lhs, rhs)` : returns `True` iff $lhs \parallel rhs$.

The helpers operate on `dict[int, int]` operands. Two timestamps with disjoint key sets should be treated as incomparable (`timestamp_le` returns `False` in either direction).

2.5.2 The `Node` Class

The `Node` class now represents a quintuple (i, t, p, m, c) rather than a triple (p, m, c), in order to support multi-value storage. The two new fields are:

- `i`: the ID of the replica that produced the version.
- `t`: the timestamp at which the version was produced (an `int` under Lamport, a `dict[int, int]` under vector).

In addition, the `metadata` dictionary now has two reserved keys that the library and the user share responsibility for:

- `"status"` : takes the value `"active"` or `"deleted"`. **The user is responsible for setting this key** when generating a `Move` operation; an operation with `"status": "deleted"` acts as a tombstone for the target node ID.
- `"applied"` : a Boolean. **The library is responsible for setting this key** when applying the operation; it records whether the operation was actually applied to the tree state (`True`) or only recorded in the log (`False`, e.g. because the operation was a Move-Wins loser as in Section 2.3).

The `timestamp` accessor of `Node`, like that of `VectorClock`, must return a deep copy when the timestamp is a dictionary. `Node` must be hashable so that instances can be stored in sets (and therefore in `frozensets`; see Section 2.5.3); a sensible `__hash__` hashes a fully canonicalised tuple of the five fields.

2.5.3 The `Tree` Class

In Phase 2, the `Tree` associates each node ID c with a *set of versions* of that node, rather than a single triple as in Phase 1. The *internal storage* you choose to back this association is up to you, exactly as it was in Phase 1: a dictionary mapping IDs to sets, a list of `Node` objects you index on demand, or any other container of your choosing is acceptable, as long as the methods specified below behave according to their contracts. The external contracts on `__call__` and `__getitem__` (below) do constrain the *return* types — they are part of the test-facing API — but those constraints do not propagate into your internal layout.

The required methods are:

- `__init__(self)`: construct an empty tree.
- `__call__(self, deleted: bool = False) -> list[frozenset[Node]]`: return the current state of the tree as a list of `frozenset`s, one per child ID, sorted by child ID. `frozenset`s rather than plain sets are used because the result must itself be hashable (a plain `set[Node]` cannot be a member of a `set`). When `deleted` is `False`, versions whose status is `"deleted"` are filtered out; when `True`, they are included.
- `__getitem__(self, key: int) -> set[Node]`: return the set of versions associated with a child ID, or the empty set if the ID is unknown.
- `__iter__(self)`: iterate over the child IDs known to the tree. Required by the test suite to inspect the per-child sets of versions without exposing the internal layout.
- `get_active(self, key: int) -> set[Node]`: return the subset of `[key]` consisting of versions that are both non-tombstone (`status` $\neq$ `"deleted"`) and not orphaned in the sense of Section 2.4.
- `move(self, i, t, p, m, c)`: apply a `Move` operation by inserting the candidate version (i, t, p, m, c) into the per-child set of versions for child c, subject to the dominance and Move-Wins rules below. The method must first reject the operation outright if it would create a cycle — i.e., if $c = p$, or if p is currently a descendant of c through any existing version in any per-child set along the path. The descendant check should be a depth-first search over the per-version parents.

  Once the cycle check passes, let F be the per-child set for c before insertion, and let `cand` denote the candidate version with vector timestamp V_{cand}. For each existing version $v \in F$ with vector timestamp V_v, compare V_v and V_{cand} under the partial order of Section 2.1 (using the static helpers on `VectorClock`):

  - If $V_v < V_{\text{cand}}$ strictly, mark v for removal: the candidate strictly supersedes v.
  - If $V_{\text{cand}} \leq V_v$, discard the candidate entirely: the existing version dominates or equals the candidate.
  - If V_v and V_{cand} are concurrent, apply Move-Wins (Section 2.3): if the candidate is a deletion and v is alive, discard the candidate; if the candidate is alive and v is a deletion, mark v for removal; otherwise keep both versions as peers.

  Finally, remove the marked-for-removal versions from F, add the candidate (unless it has been discarded), and persist the updated per-child set in your internal storage.

2.5.4 The `Replica` Class

The `Replica` class is where the dynamic decisions sit — ordering, conflict detection, the undo-do-redo sequence, and the new *log-insertion-rollback-redo* sequence introduced by Move-Wins. It must remain thread-safe, exactly as in Phase 1.

Constructor. The constructor signature gains one new argument, `num_replicas`:

> `__init__(self, id, host, main_base, listener_base, num_replicas: int)`.

The replica's clock is initialised as `VectorClock(id, num_replicas)`. The other constructor arguments and their semantics are unchanged from Phase 1.

Operation log. The operation log remains a list of $(\text{replica_id}, t, \text{old_}p, p, m, c)$ tuples; the timestamp slot, t, now holds a `dict[int, int]` under the partial order of Section 2.1. The library-private bookkeeping inside each log entry is allowed to evolve — you can, if you think you need, add new fields into the tuple without excluding the ones that are just specified — but the externally visible `log` property must return a deep copy of the $(\text{replica_id}, t, \text{old_}p, p, m, c)$ tuples.

Tree snapshot. Beyond the active `Tree` instance that holds the current state of the CRDT, the `Replica` maintains a second `Tree` instance — the *snapshot* — which holds the cumulative effect of all log entries already known to be causally stable (Section 2.7). The snapshot is used in two places:

- as the destination into which the log-compaction routine folds entries that fall below the causal-stability threshold (Section 2.7.4); and
- as the starting state from which the active tree is rebuilt during the log-insertion-rollback-redo recovery path of `__apply_move` (below).

The snapshot is part of the replica's internal mutable state and must be guarded by the same locking strategy as the active tree, the clock, and the operation log. Its externally visible representation (if you choose to expose one) must, like the other accessors, return a deep copy.

In-order check. Before applying an incoming operation `op`, you must decide whether it can be appended to the operation log without disturbing the entries already there. Compare `op`'s timestamp to the timestamp of the last log entry under the partial order of Section 2.1:

- if `op` is empty (no prior entries), `op` is in order;
- if `op` *strictly happens before* the last entry (`op.t` $<$ `last.t` under `timestamp_lt`), `op` is out of order;
- otherwise — including the concurrent case (`op.t` $\parallel$ `last.t`) — `op` is in order.

The concurrent case is the substantive change from Phase 1. Two concurrent operations on the same child do not happen-before each other and are therefore both legal next entries in the log; appending the incoming one directly preserves both versions as multi-value peers rather than triggering the more expensive recovery sequence below.

Conflict detection. You must implement a helper, named or unnamed, that returns the set of log entries that conflict with an incoming operation in the sense of Section 2.3: same child ID and concurrent timestamp. A reverse linear scan of the log is acceptable.

The `__apply_move(op)` method. This is where the central decision is made. The required behaviour is:

1. Detect conflicts. For each conflicting log entry, apply Move-Wins:
   - If the incoming op is a deletion and a conflicting entry is alive, mark the incoming op as `"applied": False`, append it to the log, and return without touching the tree.
   - If the incoming op is alive and a conflicting entry is a deletion, flip that entry's `"applied"` flag to `False` and remember that the tree must be rebuilt.
2. If no past entries had their flags flipped *and* the incoming op is in order, append the op to the log and apply it to the tree.
3. Otherwise — either the op is out of order, or a past flag was flipped — run the appropriate recovery sequence:
   - *Undo-do-redo* (Originating from Phase 1; extended in Phase 2). If only the op is out of order, locate its *insertion point* in the log — the first index at which the existing entry strictly happens-after `op` (concurrent entries do not count as happens-after, so they are skipped past and remain peers) — undo the suffix of the log that follows the insertion point, apply `op` at the insertion point, and redo the suffix. The crucial extension under Phase 2 is that *only entries with `"applied": True` are undone or redone.*
   - *Log-insertion-rollback-redo.* If a past entry's flag was flipped, reverting just the suffix is not enough — the flipped entry might lie before the insertion point. Insert the new op at its insertion point, restore the tree from a snapshot maintained by the replica (the same snapshot used by checkpointing, Section 2.7.4), and redo all still-applied entries currently in the log.
4. In both recovery sequences, finish by attempting log compaction (Section 2.7). The Move-Wins loser path of step 1 is the only path that does not finish with a compaction call, because it does not advance any causal-stability bound.

Peer-progress bookkeeping. You must maintain, per peer, the most recent timestamp you have heard from that peer, in order to compute the causal-stability threshold (Section 2.7). The required public methods are:

- `record_last_timestamp(self, replica_id, last_timestamp)`: register that peer `replica_id` was most recently observed at `last_timestamp`. Called by the listener-side of the receive path whenever a peer advertises its progress (Section 2.7.2).

- `get_peer_timestamp(self, peer_id)` : return the most recent timestamp recorded for peer `peer_id`, or the all-zeros vector with the same key set as the local clock when no timestamp has been recorded yet. The return value must be deep-copy-safe so that the caller cannot mutate internal state through it.

2.5.5 Communication Protocol Updates

The communication protocol of Phase 2 has two material changes from Phase 1: the channel used for termination (`DONE/ACK`) moves from Publish-Subscribe to Request-Reply, and a replica must now communicate to its peers its most recent local timestamp so that each peer can compute the causal-stability threshold of Section 2.7. The `MOVE` channel itself remains the Phase 1 Publish-Subscribe channel; the message format remains the Phase 1 baseline (you may extend it with additional fields as discussed under *Peer-progress mechanism* below).

The termination protocol: from PUB-SUB to REQ-REP. The Phase 1 termination protocol — in which both `DONE` and `ACK` are published on Publish-Subscribe topics — contains an edge case that can leave the system in a livelock that is structurally similar to the Two-Generals problem.

Consider a system of two replicas R_1 and R_2, with main and listener threads (M_1, L_1) and (M_2, L_2) respectively. Suppose the following interleaving occurs:

1. $M_1 \to L_2$: `DONE`; L_2 sets its local "all replicas done" event for R_2.
2. $L_2 \to M_1$: `ACK`; R_1's ACK counter for L_2 becomes 1.
3. $M_2 \to L_1$: `DONE`; L_1 sets its local event for R_1.
4. L_1 does *not* send an `ACK` back to M_2 (because of the asymmetric arrival of `DONE` messages across the Publish-Subscribe layer); R_2's ACK counter for L_1 remains at 0.

R_1 now passes its barrier and exits, while R_2 stays in a livelock awaiting an `ACK` that will never come. The deeper reason for this is that the Publish-Subscribe pattern, while implemented over TCP, deliberately discards TCP's per-message reliability and per-recipient confirmation at the application layer: a publisher cannot tell who actually received a given message. The Two-Generals problem is itself unsolvable in the worst case, but in the typical case we can avoid this particular livelock by switching to a pattern in which TCP's reliability is preserved end-to-end — namely, Request-Reply.

The fix is therefore to move `DONE` and `ACK` off the Publish-Subscribe channel and onto a per-peer Request-Reply channel, which gives every `DONE` a matching, blocking `ACK` from the specific peer it was sent to. The new socket layout is:

- *Main thread.* In addition to the `MOVE-PUB` socket from Phase 1, the main thread now owns one `DONE-REQ` socket per peer, each connected to that peer's listener address. To announce termination, the main thread sends a JSON-formatted `DONE` message (with the same object structure as the one described in Phase 1) to each peer in turn over the corresponding `DONE-REQ` socket and blocks on `recv()` for the peer's `ACK` (now a plain string `"ACK"` rather than a JSON object). The `recv()` operation must be guarded by

a timeout (e.g. 5 seconds) so that a crashed or unresponsive peer cannot deadlock the sender. The `ACK-SUB` socket of Phase 1 is removed.

- *Listener thread.* In addition to the `MOVE-SUB` socket from Phase 1, the listener thread now owns a single `DONE-REP` socket bound to the replica's listener address. It blocks (also with a timeout, as a crash-safety measure) on `recv()` for incoming `DONE` messages and replies `"ACK"` to each one. The `ACK-PUB` socket of Phase 1 is removed.

Two consequences of this change need to be reflected in the `run_replica` function:

- `ACK`s are no longer treated as idempotent at the protocol level: the listener replies with an `ACK` to every `DONE` it receives, not just the first one. The main thread must therefore deduplicate at the application layer by tracking the *set* of peer IDs from which it has received at least one `ACK`.
- Termination correctness now depends on a barrier condition that distinguishes "`ACK` received" from "timeout reached." Let `received_acks` be the set of peer IDs from which the main thread has actually received an `ACK`, and `failed_to_send_acks` be the set of peer IDs for which the timeout expired. The exit condition is

$$\texttt{all_replicas_done_event.is_set()} \wedge \\ \texttt{received_acks} \cup \texttt{failed_to_send_acks} \cup \{\texttt{self.id}\} \\ = \{0, 1, \ldots, N-1\}.$$

  Convergence to a common final tree state in the timeout-reached case is preserved by the CRDT's Strong Eventual Consistency: a peer that has already sent at least one `DONE` has stopped generating `MOVE` operations, so no message that could later diverge the final state is still in flight.

Peer-progress mechanism. A replica must maintain, per peer, the most recent timestamp it has heard from that peer (Section 2.7.2). The wire-level mechanism by which peers communicate this information is a design choice. Reasonable options include piggybacking the timestamp on every `MOVE` message (e.g. as an extra key in the `metadata` dictionary, which Phase 1 treats as opaque), or running a separate periodic broadcast on a dedicated topic.

Whatever mechanism you choose, your design must remain correct under the listener-thread/main-thread interleaving inside a replica process: the timestamp you broadcast should be your *current* clock value at broadcast time, which may differ from the timestamp of the operation you are broadcasting, because your listener may have applied a remote operation in between. You are required to document your choice in your report.

2.6 Forward and Backward Compatibility

One of the main objectives of Phase 2 is to make you reason *explicitly* about how a distributed system evolves. Every real deployment must coexist with older

versions of itself during a rollout and must avoid baking in choices that paint future versions into a corner. You will exercise both directions in this phase, but with different deliverables: backward compatibility with Phase 1 is a *report-only* discussion, while forward compatibility toward a hypothetical Phase 3 is something you should also *implement* to the extent reasonable, in addition to discussing.

2.6.1 Backward Compatibility with Phase 1 (Discussion Only)

You are **not** required to implement an actual interoperable Phase 1 $\leftrightarrow$ Phase 2 deployment. Instead, in your report, argue *theoretically* how you would design Phase 2 so that it could participate in a system containing Phase 1 (Lamport-only) replicas, and so that Phase 1 receivers would not crash or diverge on your messages. The discussion is graded; the implementation is not.

The substantive design questions you should address in the report are:

- *Vector $\rightarrow$ Lamport projection.* A Phase 1 receiver expects a totally ordered scalar timestamp, while you would be sending it a vector. Propose a projection from a vector timestamp V to a single integer that a Phase 1 receiver could ingest without breaking its Lamport monotonicity assumption, and argue in your report why the projection you chose preserves that property and what information (if any) it discards.

- *Multi-value to single-value collapse.* A Phase 1 receiver's tree expects each child to map to a single triple, while your Phase 2 tree maintains a set of versions per child. Discuss in your report what happens to that set when all peers in the system are Phase 1 — i.e., when no concurrent operations on the same child can ever arise — and argue whether any additional translation is needed at the `Tree` layer to keep a Phase 1 receiver consistent with a Phase 2 sender.

- *Wire-format extensions.* Phase 2 introduces information that Phase 1 messages did not carry — the vector-clock timestamp itself, the `metadata["status"]` field, the peer-progress data of Section 2.7.2, and so on. Identify in your report each such extension, propose a mechanism by which it could be carried on the wire so that a Phase 1 receiver does not break on it, and discuss any pitfalls your mechanism introduces (key-name collisions in the `metadata` dictionary, additional topics a Phase 1 receiver does not subscribe to, and so on). The termination protocol also changed from PUB-SUB to REQ-REP in Phase 2 (Section 2.5.5); discuss how a hybrid deployment would handle the two patterns side-by-side.

2.6.2 Forward Compatibility Toward an Unknown Phase 3 (Implementation + Report)

We are deliberately not telling you what Phase 3 will look like. Your task in this section is to look at your own Phase 2 design and answer: *which choices have I made that I would not want to undo if a Phase 3 came along?* You are then expected to **implement** as many of the resulting precautions as you can reasonably accommodate within Phase 2, and to discuss them in your report.

In particular, you should keep your Phase 2 implementation extensible along axes such as: the addition of a third concrete clock type beyond Lamport and vector, the addition of new operation types beyond `Move` and `Delete`, the addition of new fields on operations or log entries, and the addition of new message topics on the wire. The implementation cost in Phase 2 should be modest — a slightly more general data type here, a slightly more verbose abstraction there — and the report should make explicit the choices you made and why.

You are required to identify, in your report, at least two *precautions* you have implemented with Phase 3 in mind, together with their cost in Phase 2. Examples of the kind of reasoning we expect — not prescriptions — are:

- "We kept the `Clock` ABC in front of every clock access, so that a third concrete clock type could be slotted in without touching `Replica`."
- "We expressed the `"applied"` flag as a metadata key rather than promoting it to a top-level field on log entries, because Phase 3 may need to add further per-operation bookkeeping that should travel through the same channel."
- "We wrote the wire format so that a receiver who does not understand a message topic discards the message rather than blocking, in case Phase 3 adds new topics."

2.6.3 Reporting Requirements

The compatibility section of your report must contain, at minimum:

1. a theoretical discussion of backward compatibility with Phase 1 covering each of the three bullets in Section 2.6.1 (Vector→Lamport projection, multi-value to single-value collapse, and wire-format extension hygiene including the PUB-SUB/REQ-REP coexistence question);
2. one concrete walkthrough of a hypothetical Phase 1 ↔ Phase 2 message exchange (e.g. "Phase 1 replica sends MOVE; Phase 2 replica receives, lifts t to $\{i : t, \dots\}$, applies, then broadcasts; Phase 1 replica receives, uses $\max(V)\dots$");
3. a list of the forward-compatibility precautions you actually *implemented* (at least two), with the cost each precaution imposed on your Phase 2 code, and a short rationale for why each is worth that cost.

2.7 Checkpointing

Once vector clocks are in play, the operation log can grow without bound: a log entry cannot be safely discarded until it is impossible for any future incoming operation to invalidate it. The standard CRDT solution is *log compaction driven by causal stability.*

2.7.1 Causal Stability

An operation with timestamp t is *causally stable* at replica i when every operation that i will deliver from that point on has a timestamp $t' > t$. Under FIFO

and reliable broadcast assumptions, this is equivalent to "every other replica has already advanced past t." The point of causal stability is that a causally stable operation can no longer be undone — because no future incoming operation could be a predecessor of it — and therefore, neither can any operation that preceded it in the log.

2.7.2 Tracking Peer Progress

To detect causal stability, each replica must know how far each of its peers has progressed. As stated in Section 2.5.5, the mechanism by which this information is exchanged is your design choice; whatever you choose must be backward-compatible (Section 2.6.1).

Whatever the wire mechanism, on the application side you maintain a mapping T : `peer_id` $\rightarrow$ `timestamp` — "the most recent timestamp I have ever heard from this peer" — and update it on every reception. You must expose at least one method on `Replica` (e.g. `record_last_timestamp(peer, ts)`) that lets the receive path register an updated value.

A subtle point: the timestamp a sender broadcasts should be its *current* timestamp at broadcast time, not the timestamp of the operation being broadcast. The two can differ when the sender's listener thread has applied a remote operation in between the sender's main thread (a) generating its local op and (b) broadcasting it. Broadcasting the op's own timestamp would lose the advance the listener thread caused.

2.7.3 The Causal-Stability Threshold

Let T be the per-peer map of Section 2.7.2. The *causal-stability threshold* is

$$\tau \;=\; \min_{k} T[k]$$

under the partial order of Section 2.1. The min is well-defined because vector clocks form a meet-semilattice under $\leq$. An operation in your log with timestamp t is causally stable iff $t < \tau$, strictly. (If $t = \tau$, then some peer is exactly at t and might still send a concurrent op; you must not compact yet.)

If you have not yet heard from some peer, treat its contribution to the minimum as the all-zeros vector, so that the minimum is itself the all-zeros vector and no entry is causally stable. In that initial state, no compaction takes place; this is the correct behaviour.

2.7.4 Log Compaction with a Tree Snapshot

The replica maintains a second `Tree` instance, called the *snapshot*, that holds the cumulative effect of all log entries already known to be causally stable. The compaction routine is:

1. Compute the threshold τ from Section 2.7.3.
2. Walk the active log from the front. For each entry whose timestamp t satisfies $t < \tau$:
    - if the entry's metadata says `"applied": True`, fold its effect into the snapshot tree by calling the snapshot's `move` method;

- if `"applied": False`, do nothing to the snapshot — the entry is recorded but had no effect.

3. Drop those entries from the active log.

The snapshot is what the *log-insertion-rollback-redo* path of `__apply_move` (Section 2.5.4) restores from when a past entry's `"applied"` flag is flipped. The snapshot is therefore the "frozen" part of the tree state — provably immune to retroactive Move-Wins reversals — while the active log holds the "live" part that is still subject to undo and redo.

2.7.5 When Checkpointing Does Not Run

The Move-Wins loser branch of `__apply_move` (Section 2.5.4, step 1) is the one situation in which checkpointing must *not* be invoked: the incoming operation is recorded with `"applied": False` and the method returns immediately. No causal-stability bound has advanced (the threshold τ depends only on T, not on the new entry's timestamp), and the snapshot is unchanged, so calling the compactor would be wasted work — and, depending on your implementation, might double-fold the same entry into the snapshot.

3 Package Contents

The starter package you receive for Phase 2 mirrors the layout of Phase 1, with the same `src/tree_crdt` library, the same `tests` directory, the same `main.py` entry point, the same `sample_runs` folder of reference output, and the same `README.md` for setup and run instructions. The differences versus Phase 1 are:

- A new file *src/tree_crdt/clock/vector.py* containing the scaffolding for the `VectorClock` class.
- Updated scaffolding for `Tree`, `Node`, and `Replica` reflecting the multi-value and Move-Wins signatures of Section 2.5.
- New sample-run outputs under *sample_runs* showing the expected behaviour under vector clocks. *Please note that the exact number of nodes may differ between the sample run and your run in certain configurations; as in Phase 1, this situation does not imply any issues.*
- Updated `tests` that cover vector-clock comparison helpers, multi-value tree behaviour, conflict resolution, and log compaction.

Please consult *README.md* for the up-to-date project setup instructions.

4 Submission Guideline

The submissions you are going to make must include the following files and directories:

- The *src* directory under which you have completed your implementations for the `tree-crdt` library, including the new `VectorClock` class.

- The *main.py* file which you have implemented as the main program.
- A *report.pdf* document containing your discussion of forward and backward compatibility (Section 2.6), the wire-format extensions you introduced, the choice of peer-progress mechanism, and the at-least-two forward-compatibility precautions required by Section 2.6.3.

Place the files/directories listed above into a single ZIP archive named *CS_403-534_Project_Phase_2_Group_yourgroupnumber.zip*, and submit it under the assignment for `Project - Phase 2` on SUCourse+.

Please also note that a single submission per group is sufficient.

5 Grading Criteria

- **Vector Clock and `VectorClock` Helpers (14 pts):** Correct `VectorClock` behaviour and correct implementation of `timestamp_le`, `timestamp_lt`, `timestamp_eq`, and `timestamp_concurrent` for vector operands.
- **Multi-Value Tree and Orphan Detection (14 pts):** Correct multi-value storage; correct `get_active` semantics including orphan filtering; correct `__call__(deleted=...)` representation.
- **Move-Wins Conflict Resolution (16 pts):** Correct detection of conflicting operations; correct application of the policy in both directions (incoming-deletion-loses and incoming-move-causes-flip); correctness of the log-insertion-rollback-redo sequence.
- **Replica Ordering and Convergence (16 pts):** Correct behaviour of the in-order check and insertion-point search; convergence to equivalent state across replicas under both clock types.
- **Backward Compatibility with Phase 1 — Discussion Only (8 pts):** Theoretical analysis in the report covering the four bullets in Section 2.6.1, plus the concrete Phase 1 $\leftrightarrow$ Phase 2 walkthrough required by Section 2.6.3. No implementation is required for this part.
- **Forward Compatibility Toward Phase 3 — Implementation + Report (6 pts):** At least two forward-compatibility precautions *actually implemented* in your Phase 2 code, with their costs and rationale documented in the report as specified in Section 2.6.3.
- **Checkpointing (10 pts):** Correct computation of the causal-stability threshold; correct log compaction and snapshot maintenance; correct interaction between the snapshot and the rollback path.
- **Robustness, Thread-Safety, Runtime Orchestration (10 pts):** Reliable behaviour under edge cases and malformed operations; preserved tree/log invariants; thread-safe concurrent access; graceful shutdown.
- **Report Quality (6 pts):** Clarity and completeness of the discussion required by Sections 2.6.1–2.6.3.